

Module MLOps & DataOps

Rapport final professionnel

Système de détection de fraude bancaire par Machine Learning

Sujet	Industrialisation d'un produit Data/IA de classification de fraude
Dataset	Kaggle Credit Card Fraud Detection
Stack	Python, dlt, DuckDB, dbt, Dagster, MLflow, FastAPI, Streamlit, Docker
Encadrant	Pr. Mohammed AIT DAOUD
Equipe	A compléter

Année universitaire 2025–2026

Table des matières

1	Résumé Exécutif	1
1.1	Synthèse du Produit	1
1.2	Résultats Principaux	1
1.3	Couverture des Livrables	1
1.4	Constat Global	2
2	Vision du Projet	3
2.1	Problématique	3
2.2	Objectifs	3
2.3	Utilisateurs Cibles	3
2.4	Valeur Métier	3
2.5	Data Strategy	4
3	Gestion Agile	5
3.1	Organisation	5
3.2	Product Backlog	5
3.3	Planification des Sprints	6
3.3.1	Sprint 1 : Cadrage et Baseline ML	6
3.3.2	Sprint 2 : DataOps et Qualité	6
3.3.3	Sprint 3 : MLOps, Déploiement et Monitoring	6
3.4	Analyse Agile	7
4	Architecture de la Solution	8
4.1	Vue d'Ensemble	8
4.2	Composants Techniques	8
4.3	Structure du Dépôt	9
4.4	Lineage	9

5	Pipeline DataOps	10
5.1	Ingestion avec dlt	10
5.2	Stockage DuckDB	10
5.3	Transformations dbt	10
5.3.1	Staging	10
5.3.2	Mart de Features	11
5.4	Orchestration Dagster	11
5.5	Reproductibilité	11
6	Qualité des Données	12
6.1	Data Contract	12
6.2	Tests dbt	12
6.3	Tests Python	12
6.4	Dimensions de Qualité	13
6.5	Limite Qualité	13
7	Composant Machine Learning	14
7.1	Dataset et Déséquilibre	14
7.2	Préparation des Données	14
7.3	Feature Engineering	14
7.4	Modèles Comparés	15
7.5	Résultats	15
7.6	Artefacts	15
8	MLOps, Serving et Déploiement	17
8.1	Tracking MLflow	17
8.2	API FastAPI	17
8.3	Interface Streamlit	18
8.4	Monitoring	18
8.5	Déploiement Containerisé	18
9	Validation, Tests et CI/CD	19
9.1	Tests Automatisés	19
9.2	CI/CD	19
9.3	Commandes de Reproduction	19
9.4	Analyse des Risques	20

10 Conclusion et Perspectives	21
10.1 Conclusion	21
10.2 Points Forts	21
10.3 Améliorations Recommandées	21
10.4 Appréciation Finale	22
A Annexes	23
A.1 Inventaire des Documents Existants	23
A.2 Figures Disponibles	23
A.3 Commandes Utiles	23

Table des figures

7.1	Importance des variables issue des analyses du projet	16
7.2	Analyse du seuil de décision	16
A.1	Distribution des classes	24
A.2	Analyse des montants	24

Liste des tableaux

1.1	Métriques finales du modèle	2
1.2	Synthèse des livrables obligatoires	2
2.1	Utilisateurs cibles et besoins	4
3.1	Rôles Agile attendus	5
3.2	Backlog produit synthétisé	5
4.1	Composants de l'architecture	8
6.1	Analyse des dimensions de qualité	13
7.1	Matrice de confusion finale	15
8.1	Endpoints principaux	17
9.1	Risques et mesures de réduction	20

1 Résumé Exécutif

Ce rapport présente l'analyse complète du dépôt **fraud-detection**, réalisé dans le cadre du module MLOps & DataOps. Le projet vise à transformer un dataset brut de transactions bancaires en un produit Data/IA exploitable, reproductible, testable et déployable.

La solution développée couvre l'ensemble du cycle de vie attendu : ingestion DataOps, entrepôt local DuckDB, transformations dbt, contrôle qualité, entraînement Machine Learning sans fuite de données, suivi des expériences MLflow, orchestration Dagster, service de prédiction FastAPI, interface Streamlit, persistance PostgreSQL, monitoring applicatif et pipeline CI/CD.

1.1 Synthèse du Produit

Le produit final est un système de détection de fraude bancaire. Il reçoit une transaction décrite par les variables anonymisées **V1** à **V28**, **Time** et **Amount**, applique le feature engineering du projet, puis retourne :

- une prédiction binaire : transaction normale ou frauduleuse ;
- une probabilité de fraude ;
- un niveau de risque exploitable par un analyste ;
- un enregistrement historique dans PostgreSQL lorsque la base est disponible.

1.2 Résultats Principaux

Les métriques versionnées dans **fraud-detection/models/metrics.json** indiquent une performance robuste sur le jeu de test :

La matrice de confusion enregistrée est : $TN = 42\,485$, $FP = 3$, $FN = 15$, $TP = 56$. Pour un problème très déséquilibré, la précision et la PR-AUC sont plus pertinentes que l'accuracy ; le résultat montre donc un bon compromis entre réduction des faux positifs et détection des fraudes.

1.3 Couverture des Livrables

TABLE 1.1 – Métriques finales du modèle

Métrique	Valeur
Accuracy	0.9996
Precision	0.9492
Recall	0.7887
F1-score	0.8615
ROC-AUC	0.9794
PR-AUC	0.8201

TABLE 1.2 – Synthèse des livrables obligatoires

Livrable	Statut	Eléments dans le dépôt
Vision projet	Couvert	<code>docs/vision.md</code> , problématique, utilisateurs, valeur métier, stratégie data.
Gestion Agile	Couvert à finaliser	Backlog, user stories et 3 sprints dans <code>docs/agile.md</code> ; les noms des rôles restent à compléter.
Dépôt GitHub	Couvert	Code source, README, documentation, tests, CI, conteneurisation.
Pipeline DataOps	Couvert	dlt, DuckDB, dbt et Dagster présents et reliés.
Qualité des données	Couvert	Data contract YAML, tests Python, tests dbt, documentation de sources et modèles.
Machine Learning	Couvert	Préparation, entraînement, évaluation, artefacts versionnés, métriques.
MLflow	Couvert	Tracking des paramètres, métriques, artefacts et modèle enregistré si MLflow est disponible.
Déploiement	Couvert	FastAPI, Dockerfile, docker-compose avec API, Streamlit, PostgreSQL, MLflow et Dagster.
CI/CD	Couvert	GitHub Actions : imports, tests unitaires, dbt parse, build container.

1.4 Constat Global

Le dépôt dépasse le simple prototype ML : il propose une architecture industrialisable et alignée avec les exigences du module. Les principales limites concernent le caractère statique du dataset Kaggle, l'interprétabilité réduite des variables anonymisées par PCA et la nécessité de compléter les noms de l'équipe dans la partie Agile.

2 Vision du Projet

2.1 Problématique

Les fraudes bancaires sont rares par rapport au volume total des transactions, mais leur impact financier, opérationnel et réputationnel est élevé. Leur faible fréquence rend la détection difficile : un modèle naïf peut obtenir une accuracy très élevée tout en manquant la majorité des fraudes.

Le projet répond à cette problématique par un produit IA capable d'identifier automatiquement les transactions suspectes et de fournir un score de risque exploitable par des analystes fraude.

2.2 Objectifs

Les objectifs opérationnels sont les suivants :

- construire une chaîne reproductible depuis les données Kaggle jusqu'à un modèle servi par API ;
- traiter explicitement le déséquilibre de classes ;
- éviter les fuites de données lors du preprocessing, du sampling et de l'évaluation ;
- exposer le modèle via une API REST et une interface utilisateur ;
- suivre les expériences et les artefacts avec MLflow ;
- orchestrer les étapes DataOps/MLOps avec Dagster ;
- rendre le projet testable et déployable avec Docker et CI/CD.

2.3 Utilisateurs Cibles

2.4 Valeur Métier

La valeur métier du projet repose sur quatre axes :

- **Réactivité** : score de fraude disponible en temps quasi réel via `POST /predict`.

TABLE 2.1 – Utilisateurs cibles et besoins

Utilisateur	Besoin principal
Analyste fraude	Prioriser les transactions risquées et consulter le score de fraude.
Equipe risque bancaire	Suivre les indicateurs de fraude, de volumétrie et de performance.
Equipe Data/ML	Maintenir les pipelines, reproduire l'entraînement et contrôler le modèle.
Exploitant applicatif	Vérifier la disponibilité, les métriques runtime et l'historique.
Evaluateur pédagogique	Vérifier la conformité aux livrables MLOps/DataOps.

- **Priorisation** : classement implicite des transactions selon la probabilité et le niveau de risque.
- **Traçabilité** : historique des prédictions, métriques, artefacts et versions de modèle.
- **Industrialisation** : pipeline reproductible, tests, CI/CD et déploiement containerisé.

2.5 Data Strategy

La stratégie data observée dans le dépôt est structurée autour d'un flux local contrôlé :

1. **Source** : dataset Kaggle `mlg-ulb/creditcardfraud`.
2. **Cache local** : fichier `data/raw/creditcard.csv`, non destiné à être versionné.
3. **Ingestion** : chargement par `dlt` dans DuckDB.
4. **Entrepôt local** : `data/warehouse/fraud.duckdb`.
5. **Transformation** : `dbt` construit les modèles `staging` et `mart`.
6. **Contrat** : `contracts/creditcard_fraud_contract.yml`.
7. **Expérimentation** : MLflow pour paramètres, métriques, artefacts et registry.
8. **Service** : FastAPI pour les prédictions et Streamlit pour l'interface.

3 Gestion Agile

3.1 Organisation

Le dépôt fournit une documentation Agile dans `fraud-detection/docs/agile.md`. Elle définit les rôles attendus, le product backlog, les user stories et un déroulement en trois sprints. Les noms réels des membres de l'équipe restent à compléter dans le document source.

TABLE 3.1 – Rôles Agile attendus

Rôle	Responsabilité
Product Owner	Priorisation métier et validation de la valeur livrée.
Scrum Master	Facilitation des sprints et suivi des obstacles.
Data Engineer	Ingestion, DuckDB, dbt, contrats et qualité.
ML Engineer	Entraînement, évaluation, MLflow et serving.
Data Analyst	EDA, interprétation des métriques et restitution métier.

3.2 Product Backlog

TABLE 3.2 – Backlog produit synthétisé

ID	User story	Priorité	Critère d'acceptation
US-01	Soumettre une transaction et obtenir un score de fraude.	Haute	<code>POST /predict</code> retourne prédiction, probabilité et risque.
US-02	Ingérer le dataset Kaggle dans DuckDB.	Haute	dlt crée <code>raw.creditcard_transactions</code> .
US-03	Disposer d'une table transformée et documentée.	Haute	dbt construit <code>fraud_features</code> avec tests.

ID	User story	Priorité	Critère d'acceptation
US-04	Tracker les expériences ML.	Haute	MLflow contient paramètres, métriques, artefacts et modèle.
US-05	Fournir une interface simple.	Moyenne	Streamlit consomme l'API.
US-06	Orchestrer le pipeline.	Haute	Dagster matérialise ingestion, transformation, training, évaluation et monitoring.
US-07	Automatiser les tests.	Haute	GitHub Actions exécute les tests et le build.
US-08	Surveiller le service.	Moyenne	<code>/metrics</code> expose les compteurs runtime.

3.3 Planification des Sprints

3.3.1 Sprint 1 : Cadrage et Baseline ML

Objectif : livrer un premier modèle, l'API et une compréhension initiale des données. Les travaux couvrent l'EDA, le preprocessing, l'entraînement sans leakage, FastAPI et Streamlit. La review consiste en une démonstration d'une prédiction et une lecture des métriques.

3.3.2 Sprint 2 : DataOps et Qualité

Objectif : industrialiser la donnée. Les travaux couvrent le loader Kaggle, dlt, DuckDB, dbt, le contrat de données, les tests qualité et le lineage. La review démontre le flux `dlt -> DuckDB -> dbt build`.

3.3.3 Sprint 3 : MLOps, Déploiement et Monitoring

Objectif : rendre le projet exploitable de bout en bout. Les travaux couvrent MLflow, Dagster, `/metrics`, Docker/Podman Compose, GitHub Actions, la documentation et le rapport.

3.4 Analyse Agile

La démarche est cohérente avec une progression incrémentale : d'abord la valeur ML minimale, ensuite la fiabilisation DataOps, enfin l'exploitation MLOps. Pour finaliser le livrable Agile, il reste à renseigner les noms des membres, les dates de sprint et les décisions prises lors des reviews et rétrospectives réelles.

4 Architecture de la Solution

4.1 Vue d'Ensemble

L'architecture suit une chaîne DataOps/MLOps complète :

```
Kaggle -> data/raw -> dlt -> DuckDB -> dbt -> sklearn/imblearn
                                         -> MLflow -> FastAPI -> Streamlit
                                         -> PostgreSQL history
Dagster orchestre ingestion, transformation, training, evaluation et monitoring
.
```

4.2 Composants Techniques

TABLE 4.1 – Composants de l'architecture

Couche	Fichiers / outils	Rôle
Source	Kaggle <code>creditcardfraud</code>	Données transactionnelles anonymisées.
Ingestion	<code>dataops/ingestion.py</code> , <code>dlt</code>	Validation du contrat et chargement dans DuckDB.
Stockage analytique	DuckDB	Entrepôt local pour les tables raw, staging et mart.
Transformation	<code>dbt_fraud</code>	Nettoyage, déduplication, features SQL et tests.
Qualité	contrat YAML, tests dbt, tests Python	Contrôle schéma, types, nullité et valeurs admises.
Machine Learning	<code>src/train.py</code>	Entraînement, sélection et sauvegarde du modèle.
Tracking	MLflow	Expériences, métriques, artefacts et registry.
Serving	<code>api/main.py</code>	API REST de prédiction et monitoring.
Interface	<code>webapp/app.py</code>	UI Streamlit pour prédiction, batch et dashboard.

Couche	Fichiers / outils	Rôle
Persistance	PostgreSQL, SQLAlchemy	Historique des prédictions.
Orchestration	Dagster	Matérialisation des étapes pipeline.
Déploiement	Dockerfile, docker-compose	Exécution containerisée multi-services.
CI/CD	GitHub Actions	Tests, checks d'import, dbt parse, build image.

4.3 Structure du Dépôt

```

fraud-detection/
  api/           FastAPI, schemas Pydantic, chargement modele
  config/        Parametres centralises
  dataops/       Ingestion dlt vers DuckDB
  db/            SQLAlchemy et modele Prediction
  dbt_fraud/     Projet dbt, sources, staging, marts, tests
  dagster_project/ Assets Dagster
  monitoring/    Compteurs runtime et reference stats
  src/           Preprocessing, features, train, evaluate, predict
  tests/         Tests unitaires et API
  webapp/        Interface Streamlit
  models/        Artefacts de modele et metriques
  docs/          Documentation projet

```

4.4 Lineage

Le lineage documenté et implémenté est le suivant :

1. téléchargement ou cache local du dataset Kaggle ;
2. validation contractuelle du fichier CSV ;
3. ingestion dlt dans `raw.creditcard_transactions` ;
4. transformation dbt vers `stg_transactions` ;
5. construction de `fraud_features` ;
6. entraînement avec `src/train.py` ;
7. génération de `best_model.joblib`, `metrics.json`, `feature_columns.json` et `reference_stats` ;
8. enregistrement MLflow ;
9. exposition par FastAPI et consommation par Streamlit ;
10. journalisation des prédictions et métriques runtime.

5 Pipeline DataOps

5.1 Ingestion avec dlt

Le pipeline d'ingestion est implémenté dans `fraud-detection/dataops/ingestion.py`. Il charge le fichier Kaggle, applique le contrat via `assert_creditcard_contract`, puis exécute un pipeline dlt vers DuckDB.

La table cible est `raw.creditcard_transactions`. Le mode `replace` est utilisé pour garantir une reconstruction reproductible de l'état raw à partir de la source.

5.2 Stockage DuckDB

DuckDB est utilisé comme entrepôt local, léger et reproductible. Il répond au besoin pédagogique de disposer d'un stockage analytique sans dépendance à une infrastructure cloud. Le chemin configuré est `data/warehouse/fraud.duckdb`.

5.3 Transformations dbt

Le projet dbt est placé dans `fraud-detection/dbt_fraud`. Il contient deux niveaux principaux :

- `stg_transactions` : table nettoyée, typée et dédoublée ;
- `fraud_features` : table orientée ML contenant les variables originales et les features dérivées.

5.3.1 Staging

Le modèle `stg_transactions.sql` sélectionne les lignes valides, caste les types et supprime les doublons. Les règles principales sont :

- `Time`, `Amount` et `Class` non nuls ;
- `Class` limitée aux valeurs 0 et 1 ;
- `Amount` positif ou nul ;
- variables PCA castées en double.

5.3.2 Mart de Features

Le modèle `fraud_features.sql` produit des variables utiles au modèle :

- heure transactionnelle ;
- période de la journée ;
- transformation logarithmique du montant ;
- binning du montant ;
- moyenne et écart-type des variables PCA ;
- score de risque composite ;
- interactions `amount_x_v14` et `amount_x_v17`.

5.4 Orchestration Dagster

Dagster matérialise les assets suivants :

1. `download_dataset` ;
2. `ingest_data` ;
3. `dbt_transform` ;
4. `train_model` ;
5. `evaluate_model` ;
6. `monitor_model`.

Cette orchestration relie le DataOps au MLOps : la donnée validée alimente dbt, puis le pipeline d'entraînement et les métriques de monitoring.

5.5 Reproductibilité

La reproductibilité est assurée par :

- une configuration centralisée dans `config/settings.py` ;
- un `RANDOM_STATE` utilisé dans les splits, SMOTE et modèles ;
- des artefacts sauvegardés sous `models/` ;
- des commandes documentées dans le README ;
- Docker Compose pour aligner les services d'exécution.

6 Qualité des Données

6.1 Data Contract

Le contrat de données `contracts/creditcard_fraud_contract.yml` définit la structure attendue du dataset :

- **Time** : numérique, obligatoire, non nul ;
- **Amount** : numérique, obligatoire, non nul, supérieur ou égal à zéro ;
- **Class** : entier, obligatoire, valeurs admises 0 ou 1 ;
- **V1-V28** : numériques, obligatoires, non nuls.

Ce contrat explicite également les règles de qualité et le lineage attendu. Il sert de référence à l'ingestion et à la documentation du pipeline.

6.2 Tests dbt

Le fichier `dbt_fraud/models/schema.yml` déclare des tests sur les modèles dbt :

- `not_null` sur les colonnes critiques ;
- `accepted_values` sur la cible `Class` ;
- documentation des features de risque.

Ces tests contrôlent la validité minimale des tables staging et mart avant exploitation par le modèle.

6.3 Tests Python

Le dépôt contient des tests dédiés :

- `tests/test_data_contract.py` : validation du contrat de données ;
- `tests/test_dataset_loader.py` : comportement du loader dataset ;
- `tests/test_api.py` : preprocessing, feature engineering, endpoints API, métriques et erreurs attendues.

6.4 Dimensions de Qualité

TABLE 6.1 – Analyse des dimensions de qualité

Dimension	Mise en oeuvre
Complétude	Colonnes requises validées par contrat et tests dbt.
Validité	Types numériques, labels acceptés et montants positifs.
Cohérence	Déduplication au staging et feature engineering déterministe.
Intégrité	Lineage documenté de Kaggle jusqu’au service API.
Fraîcheur	Dataset statique ; la fraîcheur temps réel n’est pas applicable dans ce contexte Kaggle.
Observabilité	Métriques runtime et événements de prédiction JSONL.

6.5 Limite Qualité

Le dataset Kaggle est anonymisé et statique. La qualité métier réelle, comme la vérification client, marchand ou géographie, ne peut pas être contrôlée avec ces variables PCA. Dans un contexte bancaire réel, le contrat devrait être enrichi par des règles métier supplémentaires et des contrôles de dérive temporelle.

7 Composant Machine Learning

7.1 Dataset et Déséquilibre

Le projet utilise le dataset Kaggle Credit Card Fraud Detection, composé d'environ 284 807 transactions dont 492 fraudes. La classe positive représente environ 0,17% des observations, ce qui impose une stratégie adaptée aux classes fortement déséquilibrées.

Les variables V1 à V28 sont anonymisées par PCA. Les variables lisibles sont **Time**, **Amount** et **Class**.

7.2 Préparation des Données

Le pipeline suit les étapes suivantes :

1. chargement du dataset ;
2. nettoyage et suppression des doublons ;
3. clipping des montants extrêmes selon la logique de preprocessing ;
4. feature engineering ;
5. split stratifié train, validation et test ;
6. scaling et SMOTE uniquement dans les folds de cross-validation via `imblearn.Pipeline`.

Le point méthodologique essentiel est l'absence de data leakage : le scaler et SMOTE ne sont pas ajustés avant le split, et le test set reste intouché jusqu'à l'évaluation finale.

7.3 Feature Engineering

Le projet ajoute 9 variables dérivées aux 30 variables initiales, pour un total de 39 features attendues par le modèle. Les principales familles sont :

- variables temporelles : heure transactionnelle et période de journée ;
- variables de montant : log, bins et interactions ;
- statistiques sur les composantes PCA : moyenne et écart-type ;
- score de risque composite construit à partir de variables corrélées à la fraude.

7.4 Modèles Comparés

Le script `src/train.py` compare quatre familles :

- Logistic Regression ;
- Decision Tree ;
- Random Forest ;
- XGBoost.

La sélection se fait sur validation, en tenant compte de la PR-AUC, du F1-score validation et du F1-score de cross-validation. Le modèle final est ré-entraîné sur train + validation, puis évalué une seule fois sur le test set.

7.5 Résultats

TABLE 7.1 – Matrice de confusion finale

	Prédit normal	Prédit fraude
Réel normal	42 485	3
Réel fraude	15	56

7.6 Artefacts

Les artefacts principaux sont :

- `models/best_model.joblib` : pipeline complet sauvegardé ;
- `models/scaler.joblib` : scaler de secours ;
- `models/feature_columns.json` : colonnes attendues en prédiction ;
- `models/metrics.json` : métriques finales ;
- `models/reference_stats.json` : statistiques de référence pour monitoring si générées.

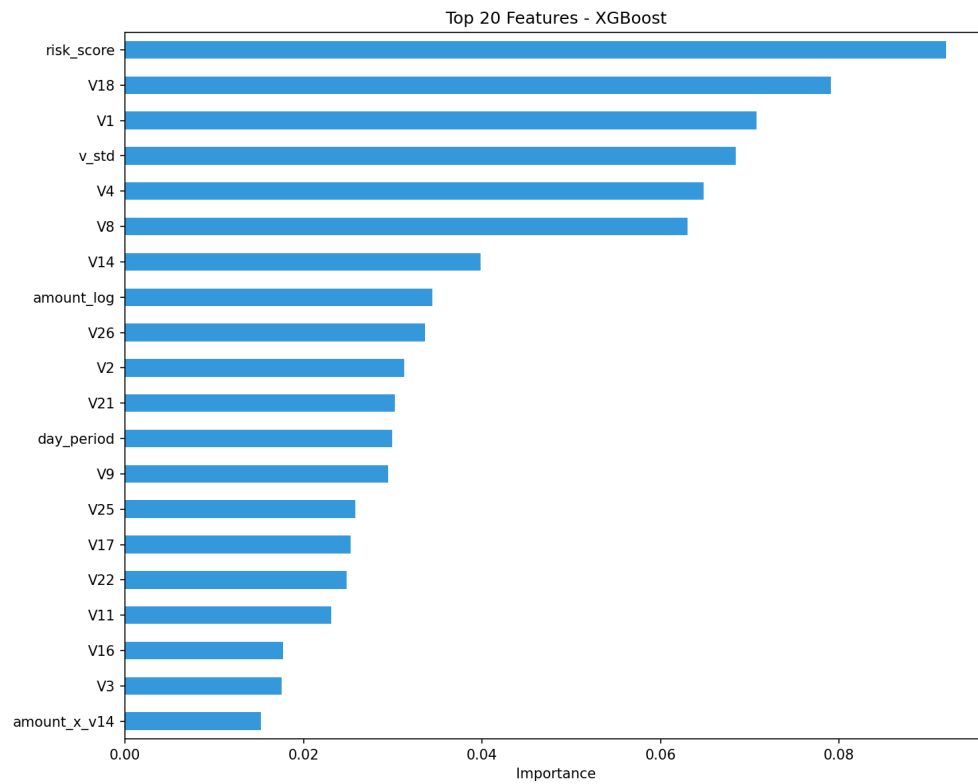


FIGURE 7.1 – Importance des variables issue des analyses du projet

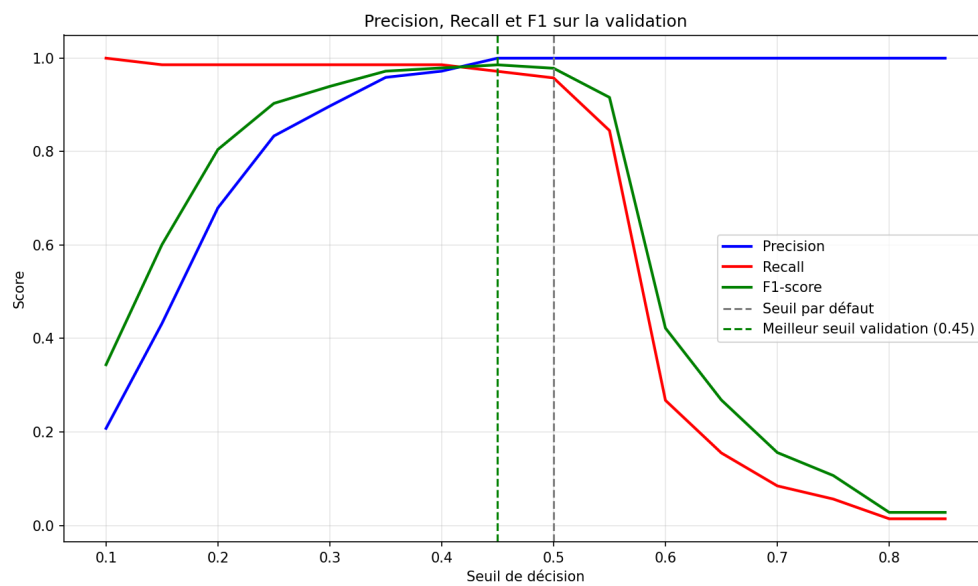


FIGURE 7.2 – Analyse du seuil de décision

8 MLOps, Serving et Déploiement

8.1 Tracking MLflow

Le tracking est intégré dans `src/train.py`. Lorsque MLflow est installé, le pipeline :

- crée ou sélectionne l'expérience configurée ;
- logge le meilleur modèle et les paramètres ;
- logge les métriques numériques de test ;
- sauvegarde les artefacts `best_model.joblib`, `feature_columns.json`, `metrics.json` et les statistiques de référence ;
- enregistre le modèle sklearn avec signature et exemple d'entrée.

8.2 API FastAPI

L'API `api/main.py` expose les endpoints suivants :

TABLE 8.1 – Endpoints principaux

Endpoint	Méthode	Rôle
/	GET	Informations générales API.
/health	GET	Statut du service et chargement modèle.
/predict	POST	Prédiction unitaire et persistance éventuelle.
/batch_predict	POST	Prédiction batch jusqu'à 1000 transactions.
/metrics	GET	Métriques Prometheus compatibles.
/model/info	GET	Métadonnées runtime du modèle.
/predictions/history	GET	Historique des prédictions en base.

La validation des entrées est assurée par Pydantic dans `api/schemas.py`. En absence de modèle chargé, l'API retourne une erreur 503 explicite. En absence de PostgreSQL, la prédiction reste disponible et l'historique retourne une liste vide.

8.3 Interface Streamlit

L'interface `webapp/app.py` consomme l'API et fournit :

- une page de prédiction unitaire ;
- un upload CSV pour prédiction batch ;
- un historique des prédictions ;
- un dashboard de synthèse.

8.4 Monitoring

Le module `monitoring/metrics.py` alimente :

- le nombre total de prédictions ;
- le taux de fraudes prédites ;
- la probabilité moyenne ;
- la latence moyenne ;
- le nombre d'erreurs ;
- la disponibilité des statistiques de référence.

Les événements de prédiction sont également enregistrables en JSONL pour faciliter une analyse ultérieure.

8.5 Déploiement Containerisé

Le fichier `docker-compose.yml` déclare cinq services :

- `api` : service FastAPI sur le port 8000 ;
- `webapp` : interface Streamlit sur le port 8501 ;
- `mlflow` : serveur MLflow sur le port 5000 ;
- `dagster` : interface Dagster sur le port 3000 ;
- `db` : PostgreSQL 16 avec healthcheck.

Cette composition couvre l'exposition minimale demandée par le module et ajoute les services MLOps utiles à la démonstration.

9 Validation, Tests et CI/CD

9.1 Tests Automatisés

Les tests unitaires et d'intégration légère couvrent les points suivants :

- nettoyage des données et scaling ;
- génération des features attendues ;
- endpoints `/`, `/health`, `/metrics`, `/model/info` ;
- erreurs API attendues : modèle absent, input invalide, batch vide ;
- contrat de données et loader dataset.

9.2 CI/CD

La pipeline GitHub Actions `.github/workflows/ci.yml` contient deux jobs :

1. **test** : installation Python 3.11, dépendances, checks d'import, tests `pytest`, `dbt parse` ;
2. **container-build** : construction de l'image Docker `fraud-detection-mlops:ci`.

Cette CI vérifie à la fois la cohérence applicative, la validité minimale du projet `dbt` et la capacité à construire l'image de déploiement.

9.3 Commandes de Reproduction

```
cd fraud-detection
python -m dataops.ingestion
cd dbt_fraud && dbt build --profiles-dir . --project-dir . && cd ..
python -m src.train
python -m src.evaluate
pytest tests/ -q
docker-compose up --build
```

9.4 Analyse des Risques

TABLE 9.1 – Risques et mesures de réduction

Risque	Impact	Mesure actuelle ou recommandée
Dataset statique	Peu représentatif d'un flux bancaire réel.	Ajouter ingestion incrémentale et tests de fraîcheur.
Variables anonymisées	Interprétabilité métier limitée.	Ajouter méthodes d'explicabilité et données métier si disponibles.
Dérive de données	Dégradation non détectée en production.	Enrichir le monitoring avec Evidently ou tests statistiques.
Seuil de décision	Coût métier non optimisé.	Calibrer le seuil avec coût FP/FN et objectifs métier.
Sécurité API	Service ouvert en démo.	Ajouter authentification, autorisation et rate limiting.
Registry local	Promotion modèle limitée.	Définir workflow staging/-production dans MLflow.

10 Conclusion et Perspectives

10.1 Conclusion

Le projet `fraud-detection` constitue une réalisation complète et cohérente pour le module MLOps & DataOps. Il ne se limite pas à entraîner un modèle : il couvre l'ingestion, la transformation, la qualité, l'entraînement contrôlé, l'évaluation, le tracking, le serving, l'interface utilisateur, la conteneurisation, l'orchestration, le monitoring et la CI/CD.

La performance finale est solide pour un problème fortement déséquilibré : F1-score de 0.8615, PR-AUC de 0.8201 et seulement trois faux positifs sur le jeu de test versionné. La méthodologie anti-leakage est un point fort du projet, notamment grâce à l'utilisation de `imblearn.Pipeline` pour isoler le scaling et SMOTE dans les folds de validation.

10.2 Points Forts

- architecture DataOps/MLOps alignée avec les attentes du module ;
- intégration réelle de dlt, DuckDB, dbt, Dagster, MLflow, FastAPI et Docker ;
- tests automatisés et CI/CD ;
- documentation structurée ;
- prise en compte du déséquilibre de classes ;
- service exploitable via API et interface Streamlit.

10.3 Améliorations Recommandées

1. compléter les noms et responsabilités réelles de l'équipe dans le livrable Agile ;
2. ajouter un suivi de drift statistique plus avancé ;
3. définir un seuil de décision optimisé par coût métier ;
4. sécuriser l'API pour un contexte de production ;
5. ajouter un workflow clair de promotion modèle dans MLflow ;
6. automatiser le réentraînement planifié dans Dagster ;
7. enrichir le rapport avec captures d'écran MLflow, Dagster et Streamlit après démonstration.

10.4 Appréciation Finale

Le dépôt présente un niveau de maturité supérieur à un projet académique minimal. Les composants sont organisés, les livrables sont majoritairement couverts, et les limites sont clairement identifiables. Après complétion des informations d'équipe et ajout de captures de démonstration, le rapport et le dépôt peuvent servir de livrable final professionnel.

A Annexes

A.1 Inventaire des Documents Existants

- docs/vision.md : vision, objectifs, utilisateurs, data strategy.
- docs/agile.md : backlog, user stories, sprint planning, reviews et rétrospectives.
- docs/architecture.md : architecture, lineage, monitoring.
- docs/final_report.md : synthèse initiale du projet.
- docs/install.md : installation.
- docs/usage.md : utilisation.
- presentation/final_presentation.md : support de présentation.

A.2 Figures Disponibles

Le dépôt contient les figures suivantes sous `fraud-detection/figures` :

- `class_distribution.png`;
- `amount_analysis.png`;
- `time_analysis.png`;
- `correlations.png`;
- `heatmap.png`;
- `new_features.png`;
- `feature_importance.png`;
- `threshold_optimization.png`;
- `error_analysis.png`.

A.3 Commandes Utiles

```
# Lancer l'API
uvicorn api.main:app --reload --port 8000

# Lancer l'interface
```



FIGURE A.1 – Distribution des classes

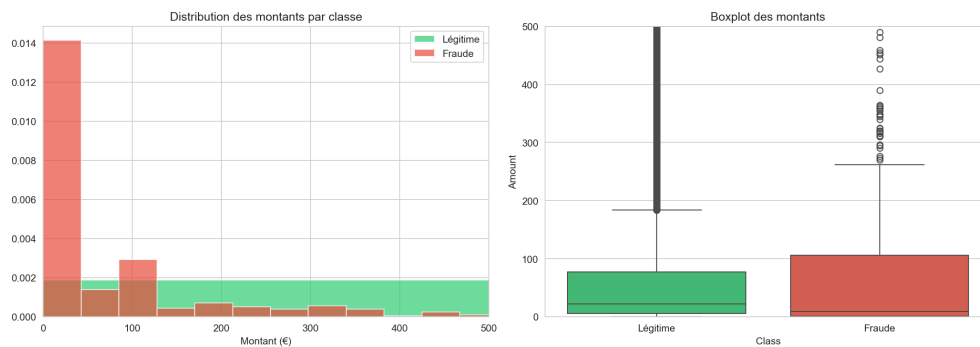


FIGURE A.2 – Analyse des montants

```
streamlit run webapp/app.py

# Lancer l'orchestration
dagster dev -m dagster_project.definitions

# Lancer l'ensemble des services
docker-compose up --build
```